certora

Security Assessment

Eigenlayer Duration Vaults

January 2026

Prepared for Eigenlayer

Table of Contents

Project Summary 3
- Project Scope 3
- Project Overview 3
- Protocol Overview 3

Assessment Methodology 5

Threat and Security Overview 6
- Security overview 6
- Design overview 6
- Threat model 8
- Centralisation risks 11
- Findings Summary 12
- Severity Matrix 12

Detailed Findings 13
- **Medium Severity Issues** 15
  - M-01: Vault rewards can become unclaimable during split activation delay 15
  - M-02: Delegation approver cannot be updated after vault initialization 16
  - M-03: Gas griefing in markMatured can permanently prevent deallocation and deregistration 18
- **Informational Severity Issues** 20
  - I-01: Duration vault operators cannot register KeyRegistrar keys 20
  - I-02: Consider emitting the value of success in _deallocateAll and _deregisterFromOperatorSet 22
  - I-03: MaxPerDepositExceedsMax error in beforeAddShares is misleading 23
  - I-04: Lock does not verify strategy is supported by operator set 24
  - I-05: VaultConfig NatSpec omits arbitrator 26
  - I-06: operatorSetRegistered state inconsistency when _deregisterFromOperatorSet reverts 27
  - I-07: Design flaw around getDurationVaults 29
  - I-08: CEI not followed in initialize allows callback before state is set 30
  - I-09: Early maturity can leave allocation state inconsistent with the vault state 31

Disclaimer 32

About Certora 32

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Eigenlayer Duration Vaults	https://github.com/Layr-Labs/eigenlayer-contracts	eb09ca4 (start commit) ddc807b (final commit)	Solidity

Project Overview

This document describes the security review of **Eigenlayer Duration Vaults**. The work was undertaken from **January 15th, 2026** to **January 28th, 2026**.

The following contract list is included in our scope:

src/contracts/strategies/DurationVaultStrategy.sol
src/contracts/strategies/DurationVaultStrategyStorage.sol
src/contracts/strategies/StrategyBase.sol
src/contracts/strategies/StrategyFactory.sol
src/contracts/core/StrategyManager.sol
src/contracts/interfaces/IDurationVaultStrategy.sol

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Duration vault strategy is a time-bound EigenLayer strategy that acts as its own operator and is designed for use cases needing fixed-term, capped stake commitments (e.g., insurance pools or financial AVSs). Vaults are deployed via StrategyFactory with a configuration that includes the underlying token, caps, duration, arbitrator, and target operator set. On initialization the vault

registers as an operator, registers for the specified operator set, and sets rewards splits so that rewards flow to stakers. Stakers must delegate to the vault before depositing, and deposits are only allowed during the open window and within per-deposit/total caps.

The vault has a simple lifecycle: DEPOSITS => ALLOCATIONS => WITHDRAWALS. The vault admin locks the vault, which blocks new deposits/withdrawal queuing and allocates full magnitude to the operator set. After the duration elapses, anyone can call markMatured() to open withdrawals and trigger best-effort deallocation/deregistration; the arbitrator can also unlock early via advanceToWithdrawals(). This design gives AVSs deterministic stake commitments while preserving EigenLayer's standard delegation, slashing, and rewards flows.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Security overview

Vault's safety depends on the correct lifecycle transitions and on the behavior of EigenLayer core contracts and AVS registrars. We focused on invariants around deposit gating and cap enforcement on active operatorShares, monotonic state transitions, and time-bounded slashability, assuming standard ERC20 behavior and honest admin/arbitrator/AVS/slasher roles. Attack vectors considered include reward split activation delay, allocation-delay edge cases, gas-griefed cleanup, delegation-approver immutability, KeyRegistrar compatibility, operator-set misconfiguration, and state divergence from best-effort deregistration, across the main value flows (deposits, allocations, rewards) and dependency surfaces (StrategyManager/AllocationManager/DelegationManager/RewardsCoordinator/registrar).

Manual review covered DurationVaultStrategy.sol and its interactions with StrategyManager hooks, AllocationManager, DelegationManager, and RewardsCoordinator across deposit, lock, maturity, and early-exit paths. Highest-severity findings were Medium, with the key takeaway to harden cleanup/retry paths and document configuration assumptions.

Design overview

DurationVaultStrategy combines strategy logic (deposits, share accounting, etc) with the operator lifecycle (registration, allocation, etc) so this single contract controls the full vault flow. The following design considerations highlight areas where additional clarity and constraints can further harden the duration vault model:

- Duration Vaults are currently meant only to be deployed by AVS, that will specifically support duration vaults, although the on-chain code is permissionless, allowing anyone to deploy a new duration vault. In the future, the design of AVSs may change and/or business decisions might make Duration Vaults open to a wider range of AVSs/deployers. We recommend documenting the current intended deployment model and its trust assumptions, and updating that documentation if governance or business decisions broaden Duration Vault usage to additional deployers/AVSs in the future.

- Implementing new strategies requires a new function to also be created in StrategyFactory & new state variables and helper functions must be implemented (i.e Duration Vaults). This also affects pre-existing functions which might not work 100% with new strategies (blacklistToken doesn't loop duration vaults). In such a case a redesign of StrategyFactory may become necessary to make it more robust and abstract or new factory implementations (both design and code wise) must be created in a way that works seamlessly with the already existing StrategyFactory to reduce code complexity.
- Special care should be taken during upgrades of the Duration Vault implementation, as the upgrade can affect duration vaults in different states (i.e some are in DEPOSITS/ALLOCATIONS/WITHDRAWAL). Any new implementation must therefore be state-agnostic and backwards-compatible: it must preserve storage layout and must not change state machine behavior in a way that can break ongoing state transitions or strand vaults in a specific state.
- Future strategies implementing beforeAddShares and beforeRemoveShares should take care to not break the main invariants of the Eigen Layer system (i.e stopping users from undelegating), unless the specific design of the implementation calls for it. Special attention should be given to beforeRemoveShares as it's called on undelegating.
- Consider reducing configuration surface area (e.g., unifying TVL limit updates into one role-aware mechanism for vaultAdmin/unpauser), and clarifying metadata conventions (vault metadataURI vs operator metadataURI) makes integrations easier to reason about.
- The vault's "time-bounded commitment" implicitly assumes specific EigenLayer timing parameters and their relative ordering (e.g., withdrawal delay, allocation delay, deallocation delay, activation delay). Since these are core-protocol values that may be adjusted over time, it's worth explicitly documenting the assumptions the duration vault relies on. Also, enforcing a minimum duration that is at least the effective allocation delay used by the vault's operator integration helps keep the commitment window.
- Some AVSs enforce a "slashable-before-register" policy and will revert during operator registration if the operator is not already slashable (e.g., allocation is not yet effective). Since the duration vault registers for the operator set during initialization but only allocates on lock(), this sequencing becomes an explicit AVS-compatibility requirement for duration vault deployments. If broader compatibility is desired, the design can instead support a flow that defers registration until after allocation is in place.

Threat model

Assets

- **Underlying ERC20 principal held by each DurationVaultStrategy**
  - Core user deposits held by vaults
- **Staker share balances and exchange-rate integrity** – determines fair withdrawals and caps
- **Operator allocations/magnitudes and time-bounded slashability guarantees**
  - Enforces limited, predictable slashing exposure
- **Reward entitlements (staker rewards + any operator-attributed rewards)**
- **Operator set registration status and AVS eligibility** – controls participation in AVS services
- **Delegation relationships and delegation approval rules** – determine who can deposit and be slashable
- **Queued-withdrawal shares and their timing relative to slashing windows**
  - Protects users before/after allocation
- **TVL caps and deposit gating invariants**
  - Limits total capital committed to vault
- **Operator metadata URIs and registration data used by AVS registrars**
  - Off-chain monitoring and operator identity

Actors

- **Vault admin –** locks, controls operations, specify caps, metadata
- **Arbitrator**
  - Set upon initialisation
  - Responsible to call advanceToWithdrawals()
- **Stakers/depositors**

- Delegate, deposit, queue/complete withdrawals
- **AVS / AVS Registrar**
  - Operator registration and operator-set membership
  - Duration vault strategy whitelisting
- **StrategyFactory owner –** whitelist/blacklist controls
- **Pausers / protocol multisig**
- **Middleware / Indexers** – rely on events and views for off-chain state reconstruction
- **Slashers**
  - Single operator set slasher can call slashOperator()
- **Griefing attacker** – gas or state bloat focused
- **Proxy admin / upgrade executor** – controls implementation + storage expectations

Trust Assumptions

- **Core EigenLayer contracts are correct and available**
- **Standard ERC20s**
  - Underlying token of duration vault strategy behaves as standard without weird traits (e.g. callbacks, fee-on-transfer, etc)
- **Slasher correctness**
  - If slashing is supported, registered operator set is assumed to have a compatible *and* functional slasher
- **Off-chain agreements around duration/insurance are enforced externally**
  - Arbitrator is trusted to unlock when required
- **Upgrade executor maintains storage layout compatibility across beacons/proxies**

Attack Vectors

- **Price-impact concern: deposit rate uses post-deposit state**
  - No exploit found; caps still enforced

- **completeQueuedWithdrawal with receiveAsTokens to be false revert during ALLOCATIONS**
- **DoS/UX risk from unbounded getDurationVaults array growth**
  - Unbounded arrays can revert view calls
- **Reward claimability failure during split activation delay**
  - Operator rewards stuck during delay window
- **Extended slashability if maturity occurs before allocation delay ends**
- **Metadata URI mismatch (vault vs operator)** – different fields by design
- **Gas griefing in markMatured can permanently prevent deallocation/deregistration**
  - Low gas skips cleanup permanently
- **Operational rigidity: delegation approver cannot be updated post-init**
- **Cap-check DoS via queued withdrawals**
  - If totalShares were used; mitigated by operatorShares
- **Rogue AVS behavior: deregister/remove strategy or ignore allocations, impacting rewards/slashability timing**
  - AVS can disable rewards eligibility
  - Slashing will be impossible
- **Donation-based cap bypass** – direct token transfers inflate exchange rate without cap checks
- **Centralization/censorship risk from StrategyFactory owner actions (whitelist/blacklist)**
  - Owner can halt deposits arbitrarily
  - Duration vault can still operate and lock with the existing deposits
- **Front-run lock by queuing withdrawals** – reduce capital actually locked
- **Paused AllocationManager / registrar callback revert can block deregistration attempts**
  - External pauses prevent operator cleanup

- **Key registration incompatibility for AVSs requiring KeyRegistrar keys**

Centralisation risks

The lifecycle of a duration vault is heavily dependent on the actions of the **Vault Admin** and **Arbitrator**. The Vault Admin can arbitrarily `lock` the vault at any given time, with no required preconditions, he is the only actor that can "start" the lifecycle of the duration vault. If he never locks the vault, users will have to queue a withdrawal and wait for the withdrawal delay to pass in order to complete their withdrawal, leaving them with an opportunity loss. He can also `lock` prior to a specific user deposit and set TVL caps to a smaller value than the current TVL in the duration vault, allowing him to deny certain users that he may not want to deposit into the vault.
The Arbitrator can call `advanceToWithdrawals` prior to the allocation being applied, thus `_deAllocateAll` will revert, but will not be caught due to the current design. This leaves the vault in a precarious position, as there is no other way to deallocate, and the operator will be fully allocated but unregistered. (Described in I-09)

Any users depositing into a duration vault **must** be aware of these risks and **must** trust the **Vault Admin** and **Arbitrator** for the correct functionality of the duration vault.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	3	3	3
Low	-	-	-
Informational	9	9	6
Total	12	12	9

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	Vault rewards can become unclaimable during split activation delay	Medium	Fixed
M-02	Delegation approver cannot be updated after vault initialization	Medium	Fixed
M-03	Gas griefing in markMatured can permanently prevent deallocation and deregistration	Medium	Fixed
I-01	Duration vault operators cannot register KeyRegistrar keys	Informational	Acknowledged
I-02	Consider emitting the value of success in _deallocateAll and _deregisterFromOperatorSet	Informational	Fixed
I-03	MaxPerDepositExceedsMax error in beforeAddShares is misleading	Informational	Fixed
I-04	Lock does not verify strategy is supported by operator set	Informational	Fixed
I-05	VaultConfig NatSpec omits arbitrator	Informational	Fixed
I-06	operatorSetRegistered state inconsistency when _deregisterFromOperatorSet reverts	Informational	Fixed

I-07	Design flaw around getDurationVaults	Informational	Acknowledged
I-08	CEI not being followed in initialize allows callback before state is set	Informational	Acknowledged
I-09	Early maturity can leave allocation state inconsistent with the vault state	Informational	Fixed

Medium Severity Issues

M-01: Vault rewards can become unclaimable during split activation delay		
Severity: Medium	Impact: High	Likelihood: Low
Files: DurationVaultStrategy.sol	Status: Fixed	

Description:

DurationVaultStrategy::initialize configures reward splits to 0% (100% to stakers) via RewardsCoordinator::setOperatorAVSSplit, RewardsCoordinator::setOperatorSetSplit, and RewardsCoordinator::setOperatorPISplit. These changes are only applied after activationDelay; before that, the default operator split (10%) can still apply.

If an AVS distributes rewards during this delay window, a non-zero operator portion may be attributed to the vault as the earner. Since DurationVaultStrategy exposes no way to set a claimer or trigger claims, such operator-attributed rewards can become effectively stuck.

Recommendations:

Consider allowing the deployer/vault admin to set a designated claimer for the vault rewards (preferably at initialization).

Customer Response:

Fixed in cbfc69e

Fix Review:

Fixed

M-02: Delegation approver cannot be updated after vault initialization

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: DurationVaultStrategy.sol	Status: Fixed	

Description:

DurationVaultStrategy registers as an operator during initialization and sets a delegationApprover via DelegationManager::registerAsOperator. The vault does not expose a function to call DelegationManager::modifyOperatorDetails, so the delegationApprover can never be updated after initialization.

```
/// @inheritdoc IDelegationManager
function modifyOperatorDetails(
    address operator,
    address newDelegationApprover
) external checkCanCall(operator) nonReentrant {
    require(isOperator(operator), OperatorNotRegistered());
    _setDelegationApprover(operator, newDelegationApprover);
}
```

The delegationApprover controls:

1. Who can delegate to the operator (signature verification)
2. Who can force undelegation of stakers

Duration vaults (like those targeting financial AVSs) with stake constraints are likely to utilize delegationApprover for constrained delegations and enforced undelegations, and may need to update delegation policies throughout the deposit timeline. While not catastrophic (users can withdraw funds), the inability to update the delegationApprover breaks the vault's operational purpose when policy changes, address rotation, or security incidents require updates, forcing costly migration to a new vault that may impact lots of stakers.

Recommendations:

Add a function in DurationVaultStrategy that calls DelegationManager::modifyOperatorDetails to allow the vault admin to update the delegationApprover. Consider access control (e.g., onlyVaultAdmin) and emit an event for transparency.

Customer Response:

Fixed in cbfc69e

Fix Review:

Fixed

M-03: Gas griefing in markMatured can permanently prevent deallocation and deregistration

Severity: Medium	Impact: High	Likelihood: Low
Files: DurationVaultStrategy.sol	Status: Fixed	

Description:

The DurationVaultStrategy::markMatured function first updates the vault state to WITHDRAWALS and then attempts to deallocate and deregister from the operator set using low-level solidity calls ignoring the success of them. An attacker can call markMatured with precisely calculated gas such that the external calls to AllocationManager::modifyAllocations and AllocationManager::deregisterFromOperatorSets run out of gas and silently fail, while the state transition still succeeds.

```
/// @notice Deallocates all magnitude from the configured operator set.
/// @dev Best-effort: failures are ignored to avoid bricking `markMatured()`.
function _deallocateAll() internal {
    // ...
    // This call is best-effort: failures should not brick `markMatured()` and
lock user funds.
    // We use a low-level call instead of try/catch to avoid wallet
gas-estimation pitfalls.
    (bool success,) = address(allocationManager)

.call(abi.encodeWithSelector(IAllocationManagerActions.modifyAllocations.selector
, address(this), params));
    success; // suppress unused variable warning
}

/// @notice Deregisters the vault from its configured operator set.
/// @dev Best-effort: failures are ignored to avoid bricking `markMatured()`.
function _deregisterFromOperatorSet() internal {
```

```
    // ...
    // This call is best-effort: failures should not brick `markMatured()` and
lock user funds.
    // We use a low-level call instead of try/catch to avoid wallet
gas-estimation pitfalls.
    (bool success,) = address(allocationManager)

.call(abi.encodeWithSelector(IAllocationManagerActions.deregisterFromOperatorSets
.selector, params));
    success; // suppress unused variable warning
}
```

Once the state is set to WITHDRAWALS, subsequent calls to markMatured become a no-op due to the early return check. Since there is no alternative function to retry deallocation or deregistration, the vault remains registered to the operator set with full magnitude allocated. This causes AllocationManager::isOperatorSlashable to return true indefinitely (since registered remains true), breaking the vault's core guarantee of time-bounded slashability.

Recommendations:

Consider adding an admin-only function to retry deallocation and deregistration.

Customer Response:

Fixed in cbfc69e

Fix Review:

Fixed

Informational Severity Issues

I-01: Duration vault operators cannot register KeyRegistrar keys

Description:

The duration vault registers itself as an operator and registers for an operator set, but it does not expose any function to register or deregister operator keys in the KeyRegistrar. Key registration requires an explicit call to KeyRegistrar::registerKey, and this contract has no path to make that call or to accept/relay a key registration request.

```
/// @inheritdoc IKeyRegistrar
function registerKey(
      address operator,
      OperatorSet memory operatorSet,
      bytes calldata keyData,
      bytes calldata signature
) external checkCanCall(operator) {
      CurveType curveType = _operatorSetCurveTypes[operatorSet.key()];
      require(curveType != CurveType.NONE, OperatorSetNotConfigured());

      // Check if the operator is already registered to the operatorSet
      require(!_operatorKeyInfo[operatorSet.key()][operator].isRegistered,
OperatorAlreadyRegistered());

      // Register key based on curve type - both now require signature
verification
      if (curveType == CurveType.ECDSA) {
            _registerECDSAKey(operatorSet, operator, keyData, signature);
      } else if (curveType == CurveType.BN254) {
            _registerBN254Key(operatorSet, operator, keyData, signature);
      } else {
            revert InvalidCurveType();
      }

      emit KeyRegistered(operatorSet, operator, curveType, keyData);
}
```

As a result, duration vault operators cannot register ECDSA/BN254 keys and are incompatible with AVSs that rely on KeyRegistrar-based operator tables or signature verification. The vault only works with AVSs that do not require keys or that use a custom registry. This is a compatibility limitation that may cause the inability of deploying and operating properly some duration vault strategies.

Recommendations:

Add vaultAdmin-gated functions that call KeyRegistrar::registerKey and KeyRegistrar::deregisterKey on behalf of the vault. If this is intentional, clearly document that duration vaults are only compatible with AVSs that do not require KeyRegistrar keys.

Customer Response:

Acknowledged
Duration vaults are designed such that the vault itself acts as the operator, but it does not perform actual operator functions requiring cryptographic keys. The vault's purpose is to manage stake commitments and allocations, not to participate in AVS tasks requiring key-based signing. Duration vaults are intentionally only compatible with AVSs that do not require KeyRegistrar-based operator tables.

Fix Review:

Customer acknowledged

I-02: Consider emitting the value of success in _deallocateAll and _deregisterFromOperatorSet

Description:

Currently the result of modifyAllocations and deregisterFromOperatorSets is ignored as per design, but there is no event to indicate if the call failed or succeeded, which makes tracking failures specifically, more difficult.

Recommendations:

Implement two new events, DeallocateAttempted & DeregisterAttempted and emit success in both.

Customer Response:

Fixed in cbfc69e

Fix Review:

Fixed

I-03: MaxPerDepositExceedsMax error in beforeAddShares is misleading

Description:

DurationVaultStrategy::beforeAddShares , the MaxPerDepositExceedsMax error has a misleading name, the cap is enforcing that each deposit doesn't exceed the maxPerDeposit cap, while the error reads: "max per deposit (cap) exceeds max (what is max?)". The naming is confusing and misleading. Note that the same error is used in _setTVLLimits and it looks like the error was made for said function, as it makes more sense in the context of the invariant it is used for.

Recommendations:

Rename MaxPerDepositExceedsMax in DurationVaultStrategy::beforeAddShares to better convey the error.

Customer Response:

Fixed in cbfc69e

Fix Review:

Fixed

I-04: Lock does not verify strategy is supported by operator set

Description:

DurationVaultStrategy::lock always allocates full magnitude to the configured operator set, but it does not verify that the duration vault strategy is included in the operator set's supported strategies (_operatorSetStrategies). In AllocationManager, strategy membership is only enforced for slashability.

```
/// @notice returns whether the operator's allocation is slashable in the given
operator set
function _isAllocationSlashable(
      OperatorSet memory operatorSet,
      IStrategy strategy,
      Allocation memory allocation,
      bool _isOperatorSlashable
) internal view returns (bool) {
      /// forgefmt: disable-next-item
      return
            // If the operator set does not use this strategy, any allocation from
it is not slashable
            _operatorSetStrategies[operatorSet.key()].contains(address(strategy))
&&
            // If the operator is not slashable by the operatorSet, any allocation
is not slashable
            _isOperatorSlashable &&
            // If there is nothing allocated, the allocation is not slashable
            allocation.currentMagnitude != 0;
}
```

As a result, a vault can transition to ALLOCATIONS while its allocation is non-slashable, which may result in the AVS ignoring the allocation for rewards or slashing. This is likely a configuration risk rather than a protocol bug, since AVSs can add/remove strategies at any time.

Recommendations:

Consider adding a sanity check in lock that the vault strategy is included in the operator set's strategies (e.g., via AllocationManager::getStrategiesInOperatorSet), or document this as a deployment/configuration assumption.

Customer Response:

Fixed in cbfc69e

Fix Review:

Fixed

I-05: VaultConfig NatSpec omits arbitrator

Description:

IDurationVaultStrategy.VaultConfig includes an arbitrator field, but the NatSpec block omits a corresponding @param entry, while similar entries exist for all other fields.

```
/// @notice Configuration parameters for initializing a duration vault.
/// @param underlyingToken The ERC20 token that stakers deposit into this vault.
/// @param vaultAdmin The address authorized to manage vault configuration.
//  #AUDIT Missing arbitrator @param.
/// @param duration The lock period in seconds after which the vault matures.
// ...
struct VaultConfig {
      IERC20 underlyingToken;
      address vaultAdmin;
      address arbitrator;
      uint32 duration;
      // ...
}
```

Recommendations:

Add a NatSpec @param arbitrator entry describing its role (e.g., address authorized to call DurationVaultStrategy::advanceToWithdrawals for early maturity).

Customer Response:

Fixed in ddc807b

Fix Review:

Fixed

I-06: operatorSetRegistered state inconsistency when _deregisterFromOperatorSet reverts

Description:

DurationVaultStrategy::operatorSetRegistered uses the local vault state (DEPOSITS / ALLOCATIONS) as a proxy for whether the strategy is registered in the AVS operator set. If _deregisterFromOperatorSet fails (e.g. due to out-of-gas, a PAUSED_OPERATOR_SET_REGISTRATION_AND_DEREGISTRATION pause, or an AVS-side revert via the registrar callback) while the vault transitions to the withdrawals state, operatorSetRegistered will return false even though the strategy remains registered in AllocationManager.

```
/// From DurationVaultStrategy.sol
/// @inheritdoc IDurationVaultStrategy
function operatorSetRegistered() public view override returns (bool) {
    return _state == VaultState.DEPOSITS || _state == VaultState.ALLOCATIONS;
}

/// From AllocationManagerView.sol
/// @inheritdoc IAllocationManagerView
function isMemberOfOperatorSet(
    address operator,
    OperatorSet memory operatorSet
) public view returns (bool) {
    return _operatorSetMembers[operatorSet.key()].contains(operator);
}
```

This leads to a state inconsistency where the reported registration status does not match the actual operator set membership in AllocationManager, which can cause confusion or mislead to off-chain monitoring or integrations relying on it.

Recommendations:

Consider having DurationVaultStrategy::operatorSetRegistered query the source of truth in AllocationManager (e.g. via AllocationManager::isMemberOfOperatorSet(address(this), _operatorSet)) instead of inferring registration from the local vault state.

Customer Response:

Fixed in cbfc69e

Fix Review:

Fixed

I-07: Design flaw around getDurationVaults

Description:

deployDurationVaultStrategy can be called repeatedly, so durationVaultsByToken[token] grows without bound. The factory acknowledges this in getDurationVaults (OOG risk on large arrays), but if any on-chain integration still uses getDurationVaults, it can become unusable once the list is large. Which sounds fine in theory, but if there are hundreds of vaults any loop over all vaults is impossible or unreliable at best; pagination would at least allow enumeration in chunks. This also bloats related bookkeeping (e.g., StrategyManager::strategyIsWhitelistedForDeposit needs entries for each vault), and de-whitelisted strategies remain in the registry, making it harder to discover only the currently-whitelisted duration vaults.

Recommendations:

Consider introducing pagination helpers (e.g., getDurationVaults(token, start, count)).

Customer Response:

Acknowledged

The getDurationVaults() function is intended for off-chain use only and includes a NatSpec comment warning about potential OOG issues with large arrays: "NOTE: Consider using the public durationVaultsByToken mapping directly for on-chain integrations to avoid potential OOG issues with large arrays." On-chain integrations should access the mapping directly with known indices. Adding pagination was considered but deemed unnecessary given the off-chain-only use case.

Fix Review:

Customer acknowledged

I-08: CEI not followed in initialize allows callback before state is set

Description:

DurationVaultStrategy::initialize performs external calls before it finishes initializing the contract state (_state = VaultState.DEPOSITS). This violates the Checks-Effects-Interactions pattern: the "effects" (setting the vault state) happen after the "interactions." A registrar callback triggered via AllocationManager.registerForOperatorSets can observe the vault in an uninitialized state and get inconsistent read results (isLocked() returns true, withdrawalsOpen() returns true). While this is read-only and not currently exploitable for state changes, it is a correctness and integration-safety issue and could become riskier if future logic relies on these views during initialization.

Recommendations:

Reorder initialization to set _state = VaultState.DEPOSITS before any external calls. This would align with CEI and remove the callback-before-state-initialized window. Otherwise, you can consider using other re-entrancy guards (like nonReentrant modifier).

Customer Response:

Acknowledged

After analysis, we determined that setting _state = VaultState.DEPOSITS before _configureOperatorIntegration() could have worse outcomes. If state is set early, registrar callbacks would see depositsOpen() == true before the operator integration is complete (registration, splits not yet configured). The current approach is more conservative: the vault only reports "ready for deposits" after everything is truly configured. The callback-before-state-initialized window only affects read-only observations, and the initializer modifier prevents actual reentrancy. This is an acceptable tradeoff.

Fix Review:

Customer acknowledged

I-09: Early maturity can leave allocation state inconsistent with the vault state

Description:

If markMatured() or advanceToWithdrawals() is called while the allocation delay is still pending, _deallocateAll() will silently fail since the low-level call will swallow the revert. This can occur if: 1. if the vault duration is shorter than the allocation delay (this is mentioned as a known issue here), or 2. if advanceToWithdrawals() is called shortly after lock() while the allocation delay is still pending. The vault is, then, moving to WITHDRAWALS state and deregisters from the operator set, yet the AllocationManager will perpetually record the prior allocation (because the pending modification hasn't cleared). This creates an on-chain state mismatch where withdrawals are open and the operator is unregistered, but the allocation will show the operator fully allocated. This may cause confusion on monitoring or any off-chain accounting from the financial AVSs.

Recommendations:

Add a function that is callable by the vault owner or arbitrator, that re-tries deallocating in case of failure of the first deallocation.

Customer Response:

Fixed in cbfc69e

Fix Review:

Fixed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.